DC Practical Codes

1. Interprocess Communication

 # server.py

```
import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("0.0.0.0", 5000))
server_socket.listen(1)
print("Server listening on port 5000...")
conn, addr = server_socket.accept()
print("Connected by", addr)
data = conn.recv(1024)
print("Received:", data.decode())
conn.sendall(b"Hello from server")
conn.close()
server_socket.close()
```

#client.py

```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))
client_socket.sendall(b"Hello from client")
data = client_socket.recv(1024)
print("Received:", data.decode())
client_socket.close()
```

2. Client Server application using RPC

#server.py

```
from xmlrpc.server import SimpleXMLRPCServer
# Define the functions to be called remotely
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
# Initialize the server on localhost at port 8000
server = SimpleXMLRPCServer(("localhost", 8000))
print("RPC Server started...")
# Register the functions so they are accessible to the client
server.register_function(add, "add")
server.register_function(subtract, "subtract")
# Run the server's main loop
server.serve_forever()
```

#client.py

```
import xmlrpc.client
# Connect to the server using its URL and port
server = xmlrpc.client.ServerProxy("http://localhost:8000/")
# Get user input
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
# Call the remote functions and print the results
print("Addition:", server.add(a, b))
print("Subtraction:", server.subtract(a, b))
```

3. Group Communication

#server.py

```
import socket, threading
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('0.0.0.0', 5000))
server.listen()
clients = []
def handle(c):
    while True:
        try:
            msg = c.recv(1024)
            [other.send(msg) for other in clients if other != c]
        except:
            clients.remove(c); break
while True:
    conn, _ = server.accept()
    clients.append(conn)
    threading.Thread(target=handle, args=(conn,)).start()
```

#client.py

```
import socket, threading
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('127.0.0.1', 5000)) # Use server's IP
def receive():
    while True: print(f"\n{client.recv(1024).decode()}")
threading.Thread(target=receive, daemon=True).start()
while True: client.send(input("You: ").encode())
```

4. Clock Synchronisation (Berkeley Algorithm)

```
import threading, time
c = [10, 12, 15]
print("Before:", c)
def sync():
    avg = sum(c) / len(c)
    offsets = [avg - t for t in c]
    for i, o in enumerate(offsets): c[i] += o
    print("Offsets:", [round(o,2) for o in offsets])
    print("After:", c)
threading.Thread(target=sync).start()
```

5. Election Algorithm (bully)

```
import threading
procs = [1,2,3,4,5,6]
failed = 5
def bully(n):
    if n == failed: return
    higher = [i for i in procs if i > n and i != failed]
    if higher:
        print(f"P{n} → ELECTION to {higher}, P{max(higher)} responds OK")
        threading.Thread(target=bully, args=(max(higher),)).start()
    else:
        print(f"P{n} → No higher process alive, P{n} is LEADER")
```

```
threading.Thread(target=bully, args=(min(procs),)).start()
```

6. Mutual Exclusion Algorithm (Mutual Exclusion using Lock (Critical Section Problem))

```
import threading, time
lock = threading.Lock()
procs = {1: 0.5, 2: 0.3, 3: 0.7}  # process: CS time
def process(i, cs_time):
    print(f"P{i} → Requesting CS")
    lock.acquire()
    print(f"P{i} → Entered CS (holds for {cs_time}s)")
    time.sleep(cs_time)
    print(f"P{i} → Leaving CS")
    lock.release()
[threading.Thread(target=process, args=(i, t)).start() for i, t in procs.items()]
```

7. Deadlock Management Algorithm (Wait-For Graph (Cycle Detection))

```
graph = {1:[2], 2:[3], 3:[1], 4:[2]}
def detect_kill(g):
    path = []
    def dfs(u):
        if u in path: return path[path.index(u):] + [u]
        path.append(u)
        for v in g.get(u, []):
            res = dfs(v)
            if res: return res
        path.pop()
    for node in list(g):
        c = dfs(node)
        if c:
            v = c[-2]
            del g[v]
            for k in g: g[k] = [x for x in g[k] if x != v]
            return f"Deadlock Cycle: {c}, Aborted Process: {v}, Recovered Graph: {g}"
    return "No Deadlock"
print(detect_kill(graph))
```

8. Load Balancing Algorithm (round robin static , least connection dynamic)

```
servers = ["S1", "S2", "S3"]
conns = {s: 0 for s in servers}
idx = 0
def round_robin():
    global idx
    s = servers[idx]; idx = (idx + 1) % len(servers); return s
def least_conn():
    s = min(conns, key=conns.get); conns[s] += 1; return s
print("Round Robin:")
[print(round_robin()) for _ in range(4)]
print("\nLeast Connections:")
[print("Assigned:", least_conn()) for _ in range(4)]
```